

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	01 October 2026
Team ID	SWTID-2026-8542
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	4 Marks

Technical Architecture:

Architecture of ComicCraft: a web-based AI comic generator built on FastAPI, Google Gemini models and Stable Diffusion, showing the local / cloud demarcation, external interfaces, data storage and ML model interfaces.

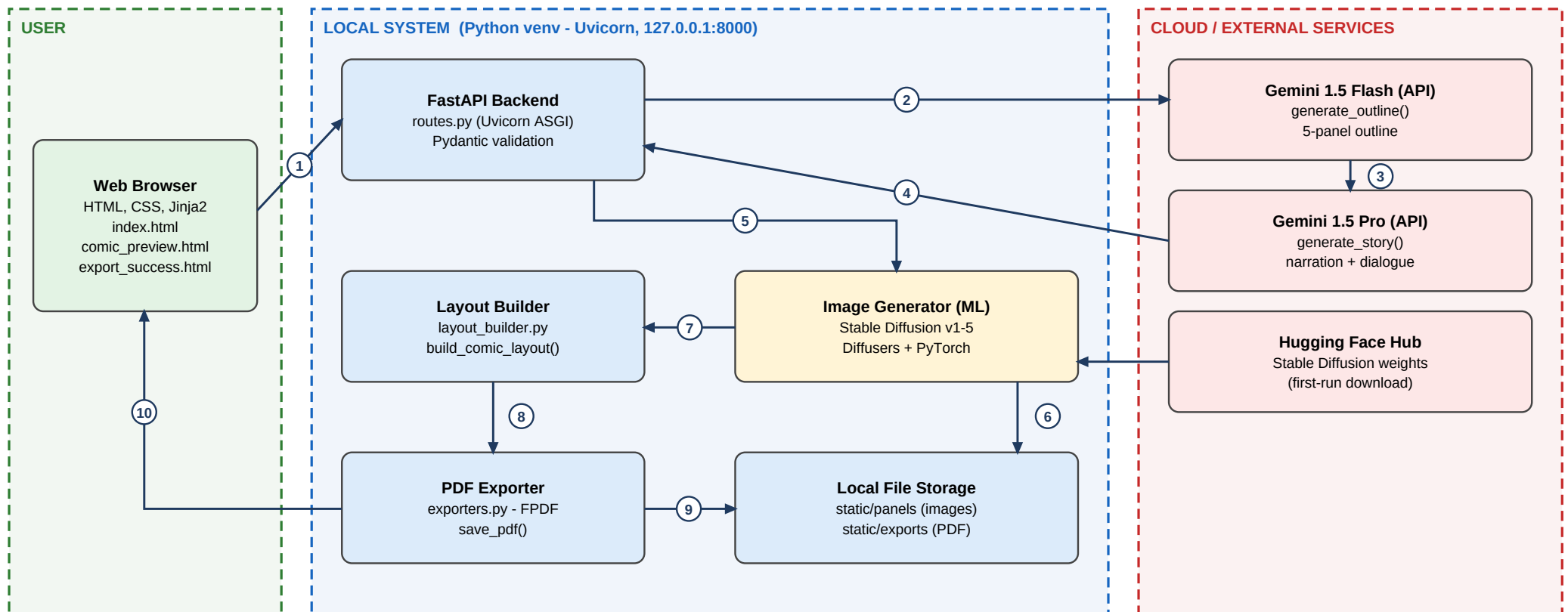


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	Web UI where users enter story prompt, character name, setting, tone and art style, preview the comic panel by panel and download it. Pages: index.html, comic_preview.html, export_success.html.	HTML, CSS, Jinja2 templates
2	Application Logic-1	Routing and input handling: form (/generate) and JSON (/generate-comic/json) requests, input validation, error handling, template rendering.	Python, FastAPI, Uvicorn, Pydantic (routes.py)
3	Application Logic-2	Generates the structured 5-panel comic outline (panel number, title, scene description, image prompt) from the user's prompt.	Google Gemini 1.5 Flash (gemini_flash.py)
4	Application Logic-3	Expands the outline into full comic narration and character dialogue for every panel.	Google Gemini 1.5 Pro (gemini_pro.py)
5	Application Logic-4	Generates a comic-style illustration for each panel from its image prompt and saves it with a sanitized filename.	Stable Diffusion via Hugging Face Diffusers (image_generator.py)
6	Application Logic-5	Layout binding of panel image, title and text, then PDF export with timestamped filename.	Python, Pillow, FPDF (layout_builder.py, exporters.py)
7	Database	No persistent database; user inputs and generated comic layout are held in memory per request and passed to the templates.	In-memory Python data structures (list of dictionaries)
8	Cloud Database	Not used in the current version.	N/A
9	File Storage	Stores generated panel images and exported comic PDFs.	Local filesystem: static/panels, static/exports
10	External API-1	Story generation: outline and narration/dialogue from Google's generative models.	Google Gemini API (google-generativeai)
11	External API-2	Source of the pretrained Stable Diffusion weights, downloaded and cached on first run.	Hugging Face Hub (diffusers, transformers)
12	Machine Learning Model	Text-to-image model for comic illustrations (runs locally), alongside the Gemini LLMs for text.	runwayml/stable-diffusion-v1-5 (PyTorch, accelerate); gemini-1.5-flash; gemini-1.5-pro
13	Infrastructure (Server / Cloud)	Application deployed on a local machine; LLM inference is delegated to Google's cloud via API. Local Server Configuration: Python virtual environment, Uvicorn on 127.0.0.1:8000 (--reload), Swagger UI at /docs; GPU recommended for Stable Diffusion. Cloud Server Configuration: Google-hosted Gemini API (API key); no self-managed cloud server.	Local (Uvicorn / virtualenv); Google Cloud (Gemini API)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Open-source frameworks and libraries used across the web layer, AI layer and export layer.	FastAPI, Uvicorn, Jinja2, Pydantic, Hugging Face Diffusers, Transformers, PyTorch, Accelerate, FPDF, Pillow
2	Security Implementations	Input validation with Pydantic and FastAPI Form parameters; exceptions handled with HTTPException and try-except blocks; prompts sanitized for safe filenames; Gemini API key kept in environment variables, not in code; HTTPS for all Gemini API calls; built-in safety filters of the AI models.	Pydantic, HTTPException, input sanitization, environment variables, HTTPS/TLS
3	Scalable Architecture	Modular 3-tier design (frontend, FastAPI backend, AI integration) with each function in its own module (outline, story, image, layout, export). Stateless route handlers allow horizontal scaling, and AI modules can be split into separate micro-services.	3-tier architecture, FastAPI, ASGI workers
4	Availability	Gemini models are served by Google's managed infrastructure. Failures in AI calls are caught and reported to the user instead of crashing the app. For production, multiple Uvicorn workers behind a load balancer can be used.	Google-managed Gemini API, Uvicorn workers, exception handling
5	Performance	Gemini Flash is used for the fast outline step and Gemini Pro only for the detailed narration. Stable Diffusion runs on GPU where available using PyTorch and accelerate; the model is loaded once and reused for all panels. Images are served as static files.	Gemini Flash, PyTorch + accelerate (GPU), static file serving

References:

<https://c4model.com/>
<https://fastapi.tiangolo.com/>
<https://ai.google.dev/>
<https://huggingface.co/docs/diffusers/en/index>
<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>